

บทที่ 1

ความรู้เบื้องต้นเกี่ยวกับซีเอสเอส

วัตถุประสงค์

- การเรียนรู้ CSS ตั้งแต่พื้นฐานไปจนถึงระดับสูง
- ประวัติการพัฒนา CSS
- คำสั่งใน CSS3
- Syntax ของ CSS3
- การแทรก CSS3
- ก่อนที่จะเริ่มเรียน CSS3
- เครื่องมือพัฒนา CSS

CSS (Cascading Style Sheets) คือ ภาษาที่ใช้สำหรับกำหนดรูปแบบและการจัดแต่งหน้าตาของเว็บไซต์ เช่น สี ฟอนต์ ขนาดของตัวอักษร การจัดตำแหน่ง หรือการจัดระเบียบองค์ประกอบต่าง ๆ บนหน้าเว็บให้ดูดีและเหมาะสมกับการแสดงผลในอุปกรณ์ต่าง ๆ CSS จะช่วยแยกส่วนการออกแบบจากเนื้อหาของเว็บไซต์ ซึ่งทำให้การพัฒนาเว็บไซต์มีความยืดหยุ่นมากขึ้น และสามารถจัดการได้ง่ายเมื่อมีการเปลี่ยนแปลงหรืออัปเดตการออกแบบ

ตัวอย่างการใช้ CSS:

```
body {  
  background-color: lightblue;  
  font-family: Arial, sans-serif;  
}
```

```
h1 {  
  color: navy;  
  text-align: center;  
}
```

CSS ทำงานร่วมกับ HTML และ JavaScript เพื่อสร้างเว็บไซต์ที่สวยงามและใช้งานง่าย โดย CSS จะเน้นไปที่การจัดรูปแบบและการแสดงผลข้อมูลที่ HTML กำหนดไว้ CSS (Cascading Style Sheets) คือ ภาษาที่ใช้ในการกำหนดลักษณะการแสดงผลของเอกสาร HTML หรือ XML เพื่อให้หน้าเว็บมีรูปแบบการแสดงผลที่สวยงามและเป็นระเบียบ โดยจะกำหนดลักษณะต่าง ๆ เช่น สี, ขนาดตัวอักษร, ตำแหน่ง, การจัดรูปแบบ และอื่น ๆ ที่เกี่ยวข้องกับการออกแบบหน้าตาของเว็บ

คำว่า **Cascading** ใน CSS หมายถึงลำดับการนำสไตล์มาใช้ หากมีการกำหนดสไตล์ในหลายที่ เช่น ในไฟล์ CSS หลายไฟล์ หรือในหลายตำแหน่งบนหน้าเว็บ CSS จะเลือกใช้สไตล์ตามลำดับความสำคัญ ซึ่งสามารถอธิบายได้ด้วยหลักการ "cascade" หรือ "ลำดับการไหล" ตัวอย่างการใช้ CSS เช่น การเปลี่ยนสีพื้นหลังของเว็บไซต์ หรือการปรับขนาดตัวอักษร โดยสามารถกำหนดได้ในหลายระดับ เช่น ในไฟล์ CSS, ภายในแท็ก `<style>` ใน HTML หรือในแอตทริบิวต์ของ HTML โดยตรง CSS ทำให้การออกแบบเว็บไซต์สามารถปรับแต่งได้ง่ายขึ้นและสามารถแยกการออกแบบจากเนื้อหาของเว็บไซต์ได้ CSS (Cascading Style Sheets) มีประวัติความเป็นมาที่สำคัญในการพัฒนาเว็บไซต์และเว็บเบราว์เซอร์ โดยเริ่มต้นจากแนวคิดการแยกการออกแบบจากเนื้อหาของเว็บไซต์ เพื่อให้การพัฒนาเว็บไซต์เป็นระบบระเบียบและง่ายขึ้น

การเรียนรู้ CSS ตั้งแต่พื้นฐานไปจนถึงระดับสูง

การเรียนรู้ CSS ตั้งแต่พื้นฐานไปจนถึงระดับสูง แบ่งหัวข้อออกเป็น 4 ระดับ ดังนี้

☐ ระดับพื้นฐาน (Beginner)

☐ รู้จัก CSS และวิธีใช้งาน

- CSS คืออะไร และทำงานอย่างไร
- วิธีเขียน CSS (Inline, Internal, External)
- โครงสร้างพื้นฐานของ CSS (Selector, Property, Value)
- หน่วยวัดใน CSS (px, %, em, rem, vh, vw)
- การกำหนดสี (color, background-color, RGBA, HEX, HSL)
- การใช้ **Typography** (font-family, font-size, line-height, letter-spacing)
- การกำหนด **Border, Margin, Padding**

☐ ระดับกลาง (Intermediate)

☐ ควบคุมเลย์เอาต์และการจัดวางองค์ประกอบ

- **Box Model** (content, padding, border, margin)
 - **Positioning** (static, relative, absolute, fixed, sticky)
 - **Display** (block, inline, inline-block, none)
 - **Flexbox** (จัดเรียงองค์ประกอบแบบยืดหยุ่น)
 - **CSS Grid** (ระบบตารางที่ทรงพลัง)
 - Pseudo-class (:hover, :focus, :nth-child)
 - Pseudo-element (::before, ::after)
 - การใช้ **Media Queries** เพื่อทำ **Responsive Design**
-

☐ ระดับสูง (Advanced)☐ เพิ่มความซับซ้อนให้กับเว็บเพจ

- การทำ **Animation** และ **Transition** (@keyframes, transition, transform)
- การใช้ **CSS Variables** (--primary-color: red;)
- การใช้ **Custom Fonts & Icons** (@font-face, Google Fonts, Font Awesome)
- การใช้ **Shadow** และ **Blur Effect** (box-shadow, text-shadow, filter)
- เทคนิคการทำ **CSS Art & Shapes**
- **Z-index** และการควบคุมลำดับชั้นขององค์ประกอบ

☐ ระดับสูงมาก (Expert)☐ ใช้ CSS อย่างมืออาชีพ

- การเขียน **CSS** ที่ยืดหยุ่นและปรับใช้ได้ง่าย (**Scalable CSS**)
- การจัดโครงสร้างไฟล์ CSS ให้อ่านง่าย (BEM, SMACSS, OOCSS)
- การใช้ **SASS / SCSS** เพื่อเพิ่มความสามารถของ CSS
- การ Optimize CSS ให้โหลดเร็ว (Minify, Critical CSS)
- เทคนิค **Dark Mode / Light Mode**
- การใช้ **CSS Houdini** เพื่อเพิ่มความสามารถใหม่ ๆ

ประวัติการพัฒนา CSS:

1. **ก่อน CSS (1989 - 1994):** ก่อนที่ CSS จะถูกพัฒนาขึ้นมา การออกแบบหน้าเว็บส่วนใหญ่จะใช้ HTML ในการกำหนดทั้งเนื้อหาและรูปแบบ (เช่น สี, ขนาดตัวอักษร, การจัดตำแหน่ง) ซึ่งทำให้เว็บไซด์ดูไม่สวยงามและปรับเปลี่ยนได้ยาก
2. **เริ่มต้นการพัฒนา CSS (1994):** ในปี 1994 **Håkon Wium Lie** (ผู้พัฒนา CSS) ได้เสนอแนวคิดของ CSS ขึ้นในระหว่างการทำงานที่ CERN (European Organization for Nuclear Research) โดยจุดประสงค์หลักของ CSS คือการแยกเนื้อหาของเว็บ (HTML) ออกจากการออกแบบ (เช่น สีและรูปแบบ) เพื่อให้สามารถจัดการและปรับแต่งเว็บไซด์ได้ง่ายขึ้น
3. **การเผยแพร่ CSS1 (1996):** CSS1 (Cascading Style Sheets Level 1) เป็นเวอร์ชันแรกที่ถูกเผยแพร่โดย W3C (World Wide Web Consortium) ซึ่งเป็นองค์กรที่ดูแลมาตรฐานเว็บ โดย CSS1 สนับสนุนการกำหนดรูปแบบพื้นฐาน เช่น สี, ขนาดตัวอักษร, การจัดตำแหน่ง และกรอบ (borders) ให้กับเนื้อหาบนหน้าเว็บ
4. **การพัฒนา CSS2 (1998):** CSS2 (Cascading Style Sheets Level 2) ถูกเผยแพร่ในปี 1998 โดยมีการเพิ่มเติมฟีเจอร์ใหม่ ๆ เช่น การจัดการเลย์เอาต์ที่ซับซ้อนมากขึ้น การใช้การวาง

ตำแหน่ง (positioning) ขององค์ประกอบในหน้าเว็บ และการสนับสนุนการทำงานกับฟอนต์และการพิมพ์ที่ยืดหยุ่น

5. **การพัฒนา CSS3 (เริ่มต้นปี 2000s):** CSS3 เป็นการปรับปรุงและขยายคุณสมบัติของ CSS2 โดยแบ่งออกเป็นหลาย ๆ โมดูล เช่น การทำงานกับแอนิเมชัน (animations), การใช้ทรานซิชัน (transitions), และกราฟิก (gradients) ซึ่งทำให้การออกแบบเว็บมีความยืดหยุ่นและทันสมัยมากขึ้น โดย CSS3 เริ่มใช้งานได้จริงในเว็บเบราว์เซอร์หลัก ๆ ตั้งแต่ปี 2010 เป็นต้นมา

6. **ปัจจุบัน:** ปัจจุบัน CSS ยังคงได้รับการพัฒนาและอัปเดตอย่างต่อเนื่อง โดยมีการเพิ่มฟีเจอร์ใหม่ ๆ ที่รองรับการใช้งานในอุปกรณ์ต่าง ๆ เช่น มือถือและแท็บเล็ต การรองรับการออกแบบที่ตอบสนอง (responsive design) และการใช้งานกับเทคโนโลยีใหม่ ๆ เช่น Flexbox และ Grid

CSS ได้รับการยอมรับและใช้งานอย่างแพร่หลายในการพัฒนาเว็บ ทำให้เว็บไซต์สามารถแสดงผลได้อย่างสวยงามและใช้งานง่ายขึ้น การนำเสนอข้างต้นแสดงให้เห็นถึงประวัติความเป็นมาของ CSS ตั้งแต่ช่วงเริ่มต้นจนถึงปัจจุบัน โดยแสดงลำดับการพัฒนาและเหตุการณ์สำคัญในแต่ละช่วงเวลา คุณสามารถดูได้ว่า CSS ได้รับการพัฒนาและขยายฟีเจอร์ต่าง ๆ อย่างไรในแต่ละเวอร์ชัน เพื่อให้การออกแบบเว็บมีความยืดหยุ่นและทันสมัยมากขึ้น CSS1, CSS2, และ CSS3 เป็นเวอร์ชันต่าง ๆ ของ CSS ที่พัฒนาเพื่อเพิ่มความสามารถและปรับปรุงการใช้งานในแต่ละเวอร์ชัน เปรียบเทียบความสามารถของแต่ละเวอร์ชันได้ดังนี้:

1. CSS1 (1996)

- **ความสามารถหลัก:**
 - กำหนดรูปแบบพื้นฐานของเนื้อหาเว็บ เช่น การตั้งค่าฟอนต์ (font), สี (color), ขนาด (size), และการจัดตำแหน่ง (alignment)
 - รองรับการตั้งค่าสีพื้นหลัง (background), กรอบ (border), และช่องว่าง (margin, padding)
 - การกำหนดสไตล์สำหรับข้อความ เช่น ขนาดตัวอักษรและการจัดแนวข้อความ
- **ข้อจำกัด:**
 - ความสามารถในการควบคุมตำแหน่งขององค์ประกอบบนหน้าเว็บจำกัดมาก
 - ไม่มีการสนับสนุนในการทำงานกับการจัดรูปแบบเลย์เอาต์ที่ซับซ้อน

2. CSS2 (1998)

- **ความสามารถหลัก:**
 - รองรับการจัดตำแหน่ง (positioning) เช่น absolute, relative, fixed, และ static ที่ทำให้สามารถวางตำแหน่งองค์ประกอบได้อย่างยืดหยุ่น
 - รองรับการใช้แถบเลื่อน (scrollbars), การตั้งค่าความกว้าง/ความสูง (width/height) แบบฟิสิกส์, และการทำงานกับกรอบ (borders) ที่ซับซ้อน
 - การรองรับการทำงานกับตัวอักษรและการพิมพ์ (font properties) ที่มีความยืดหยุ่นมากขึ้น

- การกำหนดขนาดของเอกสารและการใช้ตัวแปรต่าง ๆ สำหรับการจัดระเบียบ
- ข้อจำกัด:
 - บางฟีเจอร์ไม่ได้รับการรองรับในเบราว์เซอร์บางตัว
 - ยังคงไม่รองรับการออกแบบที่ตอบสนอง (responsive design) หรือการจัดเลย์เอาต์ที่ซับซ้อน

3. CSS3 (เริ่มต้นปี 2000s - ปัจจุบัน)

- ความสามารถหลัก:
 - การจัดการเลย์เอาต์ที่ซับซ้อน: เช่น การใช้ Flexbox และ Grid เพื่อจัดระเบียบองค์ประกอบบนหน้าเว็บได้อย่างยืดหยุ่นและแม่นยำ
 - แอนิเมชันและการเปลี่ยนแปลง: รองรับการใช้งานกับแอนิเมชัน (animations) และทรานซิชัน (transitions) ที่สามารถเปลี่ยนแปลงสไตล์ขององค์ประกอบได้แบบนุ่มนวล
 - กราฟิก: รองรับการใช้สีกราฟิกราดิเอนต์ (gradients), เงา (shadows), และการทำภาพพื้นหลังที่ซับซ้อน
 - การออกแบบที่ตอบสนอง: รองรับการสร้างเว็บที่สามารถปรับเปลี่ยนได้ตามขนาดหน้าจอ (responsive design) เช่น media queries
 - ฟอนต์: รองรับการใช้งานฟอนต์ที่ไม่ใช่ฟอนต์ในระบบโดยตรง (web fonts)
- ข้อจำกัด:
 - บางฟีเจอร์ใหม่ ๆ อาจจะยังไม่ได้รับการรองรับในเบราว์เซอร์รุ่นเก่า
 - ความซับซ้อนในการใช้ฟีเจอร์ต่าง ๆ อาจทำให้การพัฒนาเว็บไซต์ซับซ้อนขึ้น

สรุปเปรียบเทียบ:

คุณสมบัติ	CSS1	CSS2	CSS3
การตั้งค่าสี	✓ ☐	✓ ☐	✓ ☐
การตั้งค่าฟอนต์	✓ ☐	✓ ☐	✓ ☐
การจัดตำแหน่ง	จำกัด	✓ ☐ (positioning)	✓ ☐ (flexbox, grid)
กรอบและช่องว่าง	✓ ☐	✓ ☐	✓ ☐
การจัดเลย์เอาต์	จำกัด	✓ ☐ (basic layout)	✓ ☐ (advanced layout: flexbox, grid)
แอนิเมชัน	☐	☐	✓ ☐
การออกแบบที่ตอบสนอง	☐	☐	✓ ☐
การใช้กราฟิก	☐	☐	✓ ☐ (gradients, shadows, animations)
รองรับฟอนต์ภายนอก	☐	☐	✓ ☐ (web fonts)

CSS3 ได้เพิ่มคุณสมบัติและฟีเจอร์ที่ช่วยให้นักพัฒนาออกแบบเว็บไซต์ได้ทันสมัยและมีประสิทธิภาพมากขึ้น ในขณะที่ CSS1 และ CSS2 เน้นไปที่การจัดการรูปแบบพื้นฐานมากกว่า

คำสั่งใน CSS3

คำสั่งใน CSS3 สามารถแบ่งออกเป็นหลายกลุ่มตามลักษณะการใช้งานและคุณสมบัติที่ใช้ในการจัดรูปแบบหน้าเว็บ โดยทั่วไปจะสามารถแบ่งได้เป็น 6 กลุ่มหลัก ๆ ดังนี้:

1. กลุ่มการจัดตำแหน่งและเลย์เอาต์ (Layout and Positioning)

- การจัดตำแหน่ง (Positioning):
 - position: ใช้ในการกำหนดการจัดตำแหน่งขององค์ประกอบ เช่น static, relative, absolute, fixed, และ sticky
- การจัดเลย์เอาต์:
 - display: ใช้ในการกำหนดลักษณะการแสดงผลขององค์ประกอบ เช่น block, inline, inline-block, flex, grid
 - float: ใช้ในการทำให้องค์ประกอบลอยอยู่ทางซ้ายหรือขวา
 - clear: ใช้ในการเคลียร์การลอยขององค์ประกอบ
 - flex และ grid: ฟีเจอร์ใหม่ที่ใช้ในการจัดการการวางเลย์เอาต์ที่ซับซ้อน

2. กลุ่มการจัดรูปแบบข้อความ (Text Formatting)

- การตั้งค่าฟอนต์:
 - font-family: ใช้ในการกำหนดฟอนต์ของข้อความ
 - font-size: กำหนดขนาดตัวอักษร
 - font-weight: กำหนดความหนาของตัวอักษร (bold, normal)
 - font-style: กำหนดสไตล์ของฟอนต์ (normal, italic)
- การจัดรูปแบบข้อความ:
 - text-align: กำหนดการจัดตำแหน่งของข้อความ (left, right, center, justify)
 - text-transform: ใช้ในการปรับแต่งการแสดงผลของตัวอักษร (uppercase, lowercase)
 - text-decoration: กำหนดลักษณะการตกแต่งข้อความ เช่น ขีดเส้นใต้ (underline), ขีดกลาง (line-through)

3. กลุ่มการจัดการสีและพื้นหลัง (Color and Background)

- การจัดการสี:
 - color: กำหนดสีของข้อความ
 - background-color: กำหนดสีพื้นหลัง
 - border-color: กำหนดสีของขอบ
- การจัดการพื้นหลัง:
 - background-image: ใช้ในการใส่ภาพพื้นหลัง

- background-position: กำหนดตำแหน่งของภาพพื้นหลัง
- background-size: กำหนดขนาดของภาพพื้นหลัง
- background-repeat: กำหนดการทำซ้ำภาพพื้นหลัง

4. กลุ่มการเพิ่มกรอบและการจัดระยะห่าง (Borders and Spacing)

- กรอบ (Borders):
 - border: กำหนดกรอบรอบองค์ประกอบ เช่น ขนาด, รูปแบบ, และสีของกรอบ
 - border-radius: ใช้ในการสร้างมุมโค้งให้กับกรอบ
- ช่องว่าง (Spacing):
 - margin: กำหนดช่องว่างภายนอกองค์ประกอบ
 - padding: กำหนดช่องว่างภายในองค์ประกอบ

5. กลุ่มการใช้งานกราฟิก (Graphics)

- การจัดการภาพและกราฟิก:
 - box-shadow: ใช้ในการเพิ่มเงาให้กับองค์ประกอบ
 - text-shadow: ใช้ในการเพิ่มเงาให้กับข้อความ
 - linear-gradient, radial-gradient: ใช้ในการสร้างการไล่ระดับสีเป็นพื้นหลัง

6. กลุ่มการแอนิเมชันและการเปลี่ยนแปลง (Animations and Transitions)

- การเปลี่ยนแปลง (Transitions):
 - transition: ใช้ในการเปลี่ยนแปลงสไตล์ขององค์ประกอบเมื่อมีการโต้ตอบ เช่น การเปลี่ยนสีเมื่อเลื่อนเมาส์
- แอนิเมชัน (Animations):
 - @keyframes: กำหนดลำดับของแอนิเมชัน
 - animation: ใช้ในการกำหนดลักษณะของแอนิเมชัน เช่น ความยาวเวลา, การทำงานที่เสร็จสมบูรณ์, และการเล่นซ้ำ

สรุปกลุ่มคำสั่งใน CSS3:

1. **Layout and Positioning** (จัดตำแหน่งและเลย์เอาต์)
2. **Text Formatting** (การจัดรูปแบบข้อความ)
3. **Color and Background** (สีและพื้นหลัง)
4. **Borders and Spacing** (กรอบและช่องว่าง)
5. **Graphics** (กราฟิก)
6. **Animations and Transitions** (แอนิเมชันและการเปลี่ยนแปลง)

CSS3 มาพร้อมกับความสามารถใหม่ ๆ ที่ช่วยให้การออกแบบเว็บมีความยืดหยุ่นและทันสมัยมาก

ขึ้น

Syntax ของ CSS3

Syntax ของ CSS3 มีรูปแบบที่ใช้ในการกำหนดสไตล์ขององค์ประกอบใน HTML โดยทั่วไปจะประกอบไปด้วย 3 ส่วนหลัก ๆ ได้แก่ **ตัวเลือก (Selector)**, **คุณสมบัติ (Property)**, และ **ค่า (Value)** ดังนี้:

รูปแบบทั่วไปของ CSS3

```
selector {  
    property: value;  
}
```

1. **Selector** (ตัวเลือก): คือชื่อขององค์ประกอบหรือกลุ่มขององค์ประกอบที่คุณต้องการจะปรับแต่ง สไตล์ เช่น body, h1, .class-name, #id-name
2. **Property** (คุณสมบัติ): คือคุณสมบัติที่คุณต้องการจะเปลี่ยนแปลง เช่น color, font-size, margin, background-color
3. **Value** (ค่า): คือค่าที่คุณต้องการจะกำหนดให้กับคุณสมบัตินั้น ๆ เช่น red, 16px, 20px, #ffffff

ตัวอย่าง:

```
h1 {  
    color: blue;  
    font-size: 32px;  
    text-align: center;  
}
```

```
p {  
    margin: 10px;  
    font-family: Arial, sans-serif;  
}
```

ในตัวอย่างข้างต้น:

- **h1** คือ selector (เลือกทุก <h1> tag)
- **color: blue;** คือ property กับ value ที่บอกให้ข้อความใน <h1> เป็นสีน้ำเงิน
- **font-size: 32px;** คือ property กับ value ที่บอกให้ขนาดตัวอักษรใน <h1> เป็น 32px

ตัวอย่างการใช้งานใน CSS3 (การใช้ฟิเจอร์ใหม่ ๆ)

```
div {
```

```
width: 300px;
height: 200px;
background-color: lightgreen;
box-shadow: 10px 10px 15px rgba(0, 0, 0, 0.5);
border-radius: 15px;
transition: all 0.3s ease;
}
```

```
div:hover {
background-color: lightcoral;
transform: rotate(15deg);
}
```

ในตัวอย่างนี้:

- **box-shadow** คือคุณสมบัติใน CSS3 ที่ใช้เพิ่มเงาให้กับองค์ประกอบ
- **border-radius** ใช้ในการสร้างมุมโค้ง
- **transition** ใช้ในการกำหนดการเปลี่ยนแปลงเมื่อมีการโต้ตอบ (เช่น เมื่อผู้ใช้เลื่อนเมาส์ไปที่องค์ประกอบ)
- **transform** ใช้ในการหมุนหรือปรับแต่งการแสดงผลขององค์ประกอบ

ข้อควรระวัง:

- คำสั่ง CSS3 สามารถใช้งานได้ในเบราว์เซอร์ที่รองรับ (บางเบราว์เซอร์อาจยังไม่รองรับฟีเจอร์บางตัว)
- ฟีเจอร์ใหม่ ๆ เช่น **Flexbox** หรือ **Grid** อาจมีข้อแตกต่างในการรองรับขึ้นอยู่กับเวอร์ชันของเบราว์เซอร์ที่ใช้

ด้วย syntax นี้ CSS3 ช่วยให้การจัดรูปแบบเว็บง่ายและยืดหยุ่นมากขึ้น!

การแทรก CSS3

การแทรก **CSS3** ใน **HTML** สามารถทำได้หลายวิธี โดยแต่ละวิธีมีข้อดีข้อเสียแตกต่างกันไป ขึ้นอยู่กับความต้องการในการพัฒนาและการบำรุงรักษาเว็บไซต์ วิธีหลัก ๆ ในการแทรก CSS3 ใน HTML ได้แก่:

1. Inline CSS (การใช้ CSS ภายในแท็ก HTML โดยตรง)

การใช้ **inline CSS** คือการกำหนดสไตล์ในแต่ละแท็ก HTML โดยตรง การเขียน inline CSS จะใช้แอตทริบิวต์ **style** ในแท็ก HTML ซึ่งจะมีผลเฉพาะกับแท็กนั้น ๆ

ตัวอย่าง:

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Inline CSS Example</title>
</head>
<body>
  <h1 style="color: blue; font-size: 32px;">Hello, World!</h1>
  <p style="margin: 20px; color: red;">This is a paragraph with inline CSS.</p>
</body>
</html>
```

ข้อดี:

- ใช้งานง่ายและสะดวกเมื่อไม่ต้องการทำให้การจัดการสไตล์ซับซ้อน

ข้อเสีย:

- ยากในการจัดการหากเว็บไซต์มีหลายหน้า
- ไม่สามารถนำ CSS ไปใช้ซ้ำได้

2. Internal CSS (การใช้ CSS ภายในแท็ก <style> ใน <head>)

การใช้ **internal CSS** คือการวางสไตล์ทั้งหมดในไฟล์ HTML ภายในแท็ก <style> ที่อยู่ในส่วน <head> ของ HTML ซึ่งจะใช้ได้กับทั้งหน้าเว็บ

ตัวอย่าง:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal CSS Example</title>
  <style>
    h1 {
      color: blue;
      font-size: 32px;
    }
    p {
      margin: 20px;
      color: red;
    }
  </style>
</head>
<body>
```

```
<h1>Hello, World!</h1>
<p>This is a paragraph with internal CSS.</p>
</body>
</html>
```

ข้อดี:

- สะดวกในการจัดการเมื่อมีหลายองค์ประกอบที่ต้องการใช้สไตล์เดียวกันในหน้าเดียว
- ไม่ต้องแทรกสไตลลงในแต่ละแท็ก

ข้อเสีย:

- จำกัดแค่การใช้ในไฟล์ HTML เดียว ไม่สามารถใช้ซ้ำในหลาย ๆ หน้าได้

3. External CSS (การใช้ไฟล์ CSS ภายนอก)

การใช้ **external CSS** คือการแยกไฟล์ CSS ออกจากไฟล์ HTML โดยการสร้างไฟล์ .css แยกต่างหาก และลิงก์ไฟล์นั้นมาใช้ใน HTML ผ่านแท็ก <link> ในส่วน <head> ของ HTML

ตัวอย่าง:

```
<!DOCTYPE html>
<html>
<head>
  <title>External CSS Example</title>
  <link rel="stylesheet" type="text/css" href="styles.css">
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is a paragraph with external CSS.</p>
</body>
</html>
```

ไฟล์ styles.css:

```
h1 {
  color: blue;
  font-size: 32px;
}
p {
  margin: 20px;
  color: red;
}
```

ข้อดี:

- สามารถใช้สไตล์เดียวกันในหลาย ๆ หน้าได้
- ทำให้การจัดการสไตล์แยกออกจากเนื้อหาของ HTML และทำให้ง่ายต่อการบำรุงรักษา
- ลดขนาดของไฟล์ HTML

ข้อเสีย:

- ต้องแน่ใจว่าไฟล์ CSS ถูกลิงก์อย่างถูกต้องใน HTML

สรุปการใช้งาน:

- **Inline CSS:** ใช้เมื่อต้องการกำหนดสไตล์เพียงบางองค์ประกอบในหน้าเดียว
- **Internal CSS:** ใช้เมื่อมีหลายองค์ประกอบในหน้าเดียวที่ต้องการสไตล์เดียวกัน
- **External CSS:** ใช้เมื่อมีหลายหน้าเว็บที่ต้องการใช้สไตล์เดียวกัน หรือเพื่อความสะดวกในการบำรุงรักษาเว็บไซต์

โดยทั่วไปแล้ว **External CSS** เป็นวิธีที่แนะนำสำหรับเว็บไซต์ที่มีหลายหน้า เนื่องจากสามารถใช้ซ้ำและช่วยให้โค้ดง่ายต่อการบำรุงรักษามากที่สุด

ก่อนที่จะเริ่มเรียน CSS3

ก่อนที่จะเริ่มเรียน **CSS3** ควรมีความรู้เบื้องต้นเกี่ยวกับสิ่งต่าง ๆ ดังนี้:

1. HTML (HyperText Markup Language)

- **HTML** เป็นภาษาที่ใช้ในการสร้างโครงสร้างของหน้าเว็บและเนื้อหาภายในนั้น เช่น การกำหนดแท็กต่าง ๆ เช่น `<div>`, `<p>`, `<h1>`, `<a>`, `<img>`, เป็นต้น
- ความเข้าใจพื้นฐานเกี่ยวกับ HTML เช่น โครงสร้าง **HTML**, แท็ก **HTML** และ คุณสมบัติของแท็ก จะช่วยให้คุณเข้าใจวิธีการใช้ **CSS** ในการตกแต่งและจัดการการแสดงผลขององค์ประกอบต่าง ๆ บนหน้าเว็บได้ง่ายขึ้น

ตัวอย่าง:

```
<h1>This is a Heading</h1>
```

```
<p>This is a paragraph.</p>
```

2. การเข้าใจพื้นฐานเกี่ยวกับเว็บเบราว์เซอร์

- คุณควรเข้าใจว่าคำสั่ง **CSS** จะถูกประมวลผลและแสดงผลในเบราว์เซอร์ เช่น **Chrome**, **Firefox**, **Edge**, **Safari** ซึ่งแต่ละเบราว์เซอร์อาจมีลักษณะการแสดงผลที่ต่างกันในส่วนฟีเจอร์ของ **CSS**
- ความเข้าใจพื้นฐานเกี่ยวกับการใช้งานเครื่องมือพัฒนาในเบราว์เซอร์ (Developer Tools) จะช่วยในการทดสอบและแก้ไขสไตล์ **CSS** บนเว็บไซต์

3. ความเข้าใจเกี่ยวกับการแสดงผลของเว็บ (Rendering)

- รู้เกี่ยวกับการแสดงผลของเว็บในเบราว์เซอร์ เช่น การจัดระเบียบข้อมูลในหน้าเว็บ (layout) โดยใช้ **box model** ซึ่งเป็นการแสดงผลที่ประกอบด้วย **content**, **padding**, **border**, และ **margin**

- ความเข้าใจใน **document flow** หรือการจัดเรียงองค์ประกอบบนหน้าเว็บจะช่วยให้คุณใช้ CSS ได้อย่างมีประสิทธิภาพ

4. พื้นฐานเกี่ยวกับสี (Color)

- คุณควรเข้าใจวิธีการใช้สีใน HTML และ CSS เช่น การใช้ค่าสีในรูปแบบต่าง ๆ เช่น **RGB**, **Hexadecimal**, **HSL**, หรือ **Named Colors**
- ตัวอย่างเช่น `color: red;`, `color: #FF5733;`, `color: rgb(255, 87, 51);`

5. พื้นฐานการคำนวณขนาดและหน่วย (Units and Measurements)

- ความเข้าใจเกี่ยวกับ หน่วยวัด ต่าง ๆ ใน CSS เช่น **px**, **em**, **rem**, **%**, **vw**, **vh** เป็นต้น
- **px** คือพิกเซล, **em** และ **rem** ใช้ในการคำนวณขนาดที่สัมพันธ์กับขนาดฟอนต์หรือขนาดพื้นฐานของเอกสาร
- **%** ใช้ในการกำหนดขนาดที่สัมพันธ์กับองค์ประกอบหรือหน้าจอ

6. การเข้าใจเกี่ยวกับการจัดตำแหน่ง (Positioning)

- ควรเรียนรู้วิธีการจัดตำแหน่งองค์ประกอบบนหน้าเว็บ โดยใช้คุณสมบัติ **position** เช่น **static**, **relative**, **absolute**, **fixed**, และ **sticky**
- ความเข้าใจเกี่ยวกับการจัดการการแสดงผลขององค์ประกอบที่ลอย (floating) ด้วยคุณสมบัติ **float** จะช่วยให้การใช้งาน CSS3 ง่ายขึ้น

7. การเข้าใจพื้นฐานเกี่ยวกับการทำงานของฟอนต์ (Fonts and Typography)

- คุณควรมีความรู้เบื้องต้นเกี่ยวกับการใช้ฟอนต์ เช่น การตั้งค่าฟอนต์ของข้อความ (font-family), ขนาดฟอนต์ (font-size), ความหนาของฟอนต์ (font-weight), และการจัดตำแหน่งข้อความ (text-align)
- การใช้ฟอนต์ในเว็บสามารถทำได้ผ่าน **web fonts** ซึ่งจะทำให้ฟอนต์ที่ใช้ในเว็บไซต์ไม่จำกัดแค่ฟอนต์ที่มีในเครื่องเท่านั้น

8. การใช้เครื่องมือพัฒนา (Developer Tools)

- การใช้เครื่องมือของเบราว์เซอร์ เช่น **Developer Tools** (F12) สามารถช่วยให้คุณสามารถดูโค้ด CSS ที่ถูกประมวลผลและตรวจสอบการแสดงผลของเว็บไซต์ได้

สรุป: ก่อนที่คุณจะเริ่มเรียน CSS3 ควรมีความเข้าใจเกี่ยวกับ **HTML** และโครงสร้างของหน้าเว็บ, การแสดงผลของเว็บเบราว์เซอร์, การจัดระเบียบหน้าเว็บด้วย **box model**, การใช้สีและขนาดใน CSS, และวิธีการจัดตำแหน่งและการใช้ฟอนต์เบื้องต้น ซึ่งจะช่วยให้คุณเรียนรู้ CSS3 ได้ง่ายขึ้นและนำไปใช้งานได้มีประสิทธิภาพ

เครื่องมือพัฒนา CSS

เครื่องมือพัฒนา **CSS** มีหลายตัวที่ช่วยในการออกแบบและพัฒนาเว็บไซต์ได้อย่างมีประสิทธิภาพ โดยเครื่องมือเหล่านี้สามารถช่วยในกระบวนการต่าง ๆ ตั้งแต่การเขียนโค้ด, ทดสอบ, แก้ไขสไตล์, หรือแม้กระทั่งในการตรวจสอบความเข้ากันได้ของ CSS ในเบราว์เซอร์ต่าง ๆ ดังนี้:

1. Developer Tools ในเบราว์เซอร์

เครื่องมือพัฒนาในตัวเบราว์เซอร์เช่น **Chrome DevTools**, **Firefox Developer Tools**, และ **Edge Developer Tools** ช่วยให้คุณสามารถทดสอบ, แก้ไข, และตรวจสอบโค้ด CSS โดยไม่ต้องรีโหลดหน้าเว็บใหม่

- **Chrome DevTools**: กด F12 หรือคลิกขวาแล้วเลือก "Inspect" เพื่อเปิดเครื่องมือพัฒนา
- **Firefox Developer Tools**: กด F12 หรือคลิกขวาแล้วเลือก "Inspect" เช่นกัน
- **Edge Developer Tools**: กด F12 หรือคลิกขวาแล้วเลือก "Inspect" เช่นกัน

ฟีเจอร์หลักของ Developer Tools:

- **Live Editing**: แก้ไข CSS ได้ทันทีและเห็นการเปลี่ยนแปลงบนหน้าเว็บ
- **Element Inspector**: ตรวจสอบและแก้ไข HTML และ CSS ของแต่ละองค์ประกอบบนหน้าเว็บ
- **Box Model**: ตรวจสอบขนาดขององค์ประกอบและการจัดการขอบ (margins, paddings, borders)
- **Color Picker**: เลือกสีได้ง่าย ๆ ในเครื่องมือ

2. CSS Preprocessors (Sass, LESS, Stylus)

CSS Preprocessors ช่วยให้การเขียน CSS มีความยืดหยุ่นมากขึ้น เช่น การใช้ตัวแปร, การนำโค้ดมาใช้งานซ้ำ, และการใช้ฟังก์ชันในการคำนวณ

- **Sass**: ช่วยให้อ่าน CSS ได้ง่ายและมีความยืดหยุ่นมากขึ้น เช่น การใช้ตัวแปร, Mixins, และ Nested Rules
- **LESS**: คล้ายกับ Sass ใช้ตัวแปรและฟังก์ชันในการช่วยปรับแต่ง CSS
- **Stylus**: มีความยืดหยุ่นสูงและสามารถเขียนโค้ดที่เรียบง่ายกว่ามาก

ตัวอย่างการใช้ Sass:

```
$primary-color: #333;
body {
  color: $primary-color;
}
```

3. CSS Frameworks

CSS Frameworks ช่วยให้การออกแบบเว็บไซต์ง่ายและเร็วขึ้น เนื่องจากมีชุดสไตล์ที่เตรียมไว้ให้แล้ว เช่น **Bootstrap**, **Tailwind CSS**, **Foundation**

- **Bootstrap**: Framework ยอดนิยมนี่มาพร้อมกับคอมโพเนนต์ UI เช่น ปุ่ม, เมนู, การจัดเลย์เอาต์ที่ตอบสนอง (responsive)

- **Tailwind CSS:** Utility-first CSS framework ที่ช่วยให้คุณสามารถสร้างสไตล์ได้อย่างยืดหยุ่นและเร็ว
- **Foundation:** Framework ที่มาพร้อมกับเครื่องมือในการออกแบบเว็บไซต์ที่ตอบสนอง

4. Code Editors (Text Editors)

การใช้ **code editors** ที่รองรับการพัฒนา CSS สามารถช่วยให้การเขียนโค้ดสะดวกขึ้น เช่น **Visual Studio Code, Sublime Text, Atom, Brackets**

- **Visual Studio Code (VS Code):** รองรับการพัฒนา CSS ได้ดีและสามารถติดตั้งส่วนเสริม (Extensions) เช่น **Live Server** ที่ช่วยให้การทดสอบการเปลี่ยนแปลง CSS ได้ทันที
- **Sublime Text:** Editor ที่เบาและรวดเร็ว พร้อมทั้งการใช้ **Package Control** เพื่อติดตั้งส่วนเสริมต่าง ๆ ที่ช่วยในการพัฒนา
- **Atom:** Editor ที่พัฒนาโดย GitHub รองรับปลั๊กอินที่ช่วยในการพัฒนา CSS ได้ดี

5. CSS Grid and Flexbox Generators

เครื่องมือที่ช่วยในการสร้าง CSS Grid และ Flexbox layout โดยไม่ต้องเขียนโค้ดเอง

- **CSS Grid Generator:** เครื่องมือช่วยในการสร้างโค้ดสำหรับ CSS Grid layout โดยการลากและวาง
- **Flexbox Froggy:** เกมที่ช่วยสอนการใช้ **Flexbox** โดยการจัดการกับสไตล์ขององค์ประกอบในเกม
- **Flexbox Generator:** เครื่องมือออนไลน์ที่ช่วยสร้างโค้ด Flexbox layout อย่างง่าย

6. Autoprefixer

Autoprefixer คือเครื่องมือที่ช่วยในการเพิ่มพรีฟิกซ์ของเบราว์เซอร์ให้โดยอัตโนมัติ เช่น -webkit-, -moz-, -ms- เพื่อให้ CSS ทำงานได้ในเบราว์เซอร์ต่าง ๆ ที่ไม่รองรับคุณสมบัตินั้น ๆ โดยตรง

- สามารถใช้ผ่าน **PostCSS** หรือ **Gulp/Grunt** ในกระบวนการ build ของโครงการ

7. Online CSS Minifiers

การใช้เครื่องมือสำหรับการ **minify** (ย่อขนาด) CSS ช่วยให้ไฟล์ CSS มีขนาดเล็กลง ซึ่งจะช่วยให้การเพิ่มความเร็วในการโหลดหน้าเว็บ

- **CSS Minifier:** เครื่องมือออนไลน์ที่ช่วยในการย่อขนาดไฟล์ CSS โดยการลบช่องว่าง, คอมเมนต์, และปรับปรุงโค้ด CSS ให้สั้นลง

8. Browser Testing Tools

เครื่องมือทดสอบเพื่อดูว่า CSS ทำงานได้ถูกต้องในเบราว์เซอร์ต่าง ๆ เช่น **BrowserStack** หรือ **CrossBrowserTesting**

- **BrowserStack:** ให้คุณทดสอบเว็บไซต์ของคุณในเบราว์เซอร์ต่าง ๆ แบบเรียลไทม์
- **CrossBrowserTesting:** เครื่องมือที่ช่วยให้คุณทดสอบเว็บของคุณในหลากหลายเบราว์เซอร์และระบบปฏิบัติการ

สรุปเครื่องมือพัฒนา CSS:

1. **Developer Tools** ในเบราว์เซอร์ (Chrome DevTools, Firefox Developer Tools)
2. **CSS Preprocessors** (Sass, LESS, Stylus)
3. **CSS Frameworks** (Bootstrap, Tailwind CSS, Foundation)
4. **Code Editors** (Visual Studio Code, Sublime Text, Atom)
5. **Grid and Flexbox Generators**
6. **Autoprefixer**
7. **Online CSS Minifiers**
8. **Browser Testing Tools** (BrowserStack, CrossBrowserTesting)

เครื่องมือเหล่านี้จะช่วยให้การพัฒนา CSS ของคุณมีประสิทธิภาพและรวดเร็วมากขึ้น

สรุป

CSS (Cascading Style Sheets) คือภาษาที่ใช้สำหรับตกแต่งและควบคุมลักษณะของหน้าเว็บที่สร้างด้วย HTML หรือ XML โดยมีพื้นฐานการเขียนประกอบด้วย selector ที่ใช้ระบุองค์ประกอบใน HTML และ declaration ที่กำหนดค่าต่าง ๆ เช่น สี ขนาด ฟอนต์ การจัดวาง และการจัดตำแหน่ง ซึ่งสามารถแทรกในเอกสารได้ 3 แบบคือ inline, internal และ external CSS ส่วนการจัดการเลย์เอาต์จะอิงตาม Box Model ที่ประกอบด้วย content, padding, border และ margin นอกจากนี้ CSS ยังมีการจัดตำแหน่ง (positioning) หลากหลายแบบ เช่น static, relative, absolute, fixed และ sticky เพื่อควบคุมการวางองค์ประกอบบนหน้าเว็บได้อย่างแม่นยำ

CSS3 ได้เพิ่มความสามารถใหม่ ๆ เช่น transitions, animations, Flexbox, Grid Layout และ media queries เพื่อช่วยให้การออกแบบเว็บไซต์ตอบสนองและสวยงามในทุกอุปกรณ์ การใช้ selectors ต่าง ๆ เช่น element, class, ID, attribute, pseudo-class และ pseudo-element ช่วยในการกำหนดสไตล์ได้อย่างยืดหยุ่น พร้อมทั้งการจัดการฟอนต์และสีที่ช่วยให้หน้าเว็บน่าสนใจมากขึ้น การใช้ CSS ทำให้การออกแบบแยกจากเนื้อหาและทำให้เว็บไซต์ดูสวยงาม ใช้งานง่าย และสามารถบำรุงรักษาได้สะดวก.